



Module 3. Autoencoders

Basics

Deterministic Autoencoders

Variational Autoencoders

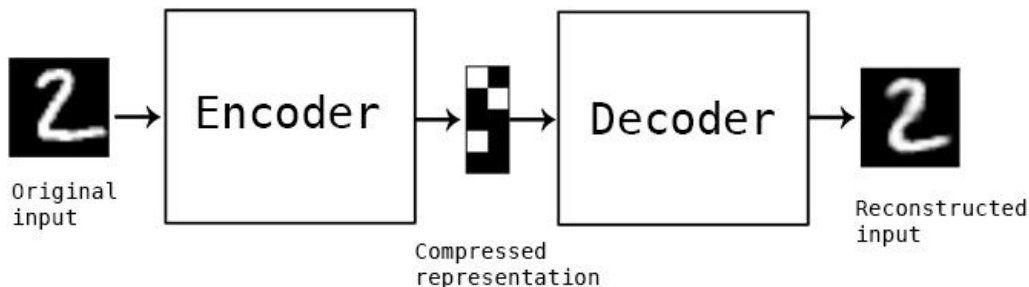
Applications



Basics

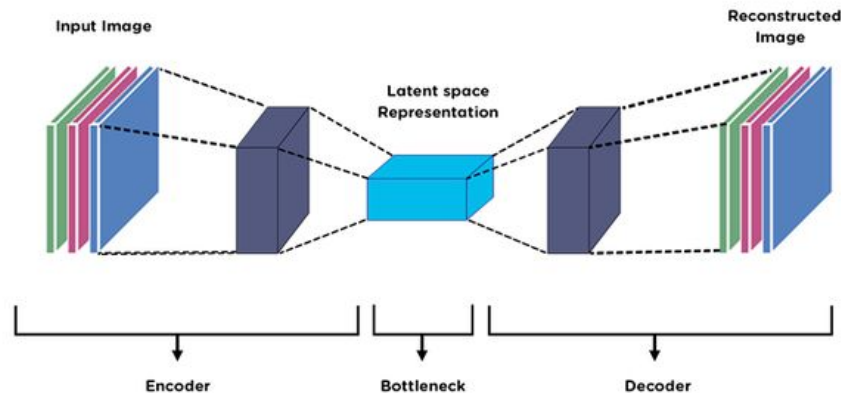
What are Autoencoders?

- Autoencoders are neural networks designed to learn efficient representations of input data → with no need for labels (**Unsupervised**)
 - Also called *Auto-Associative Neural Network*
- They consist of an **encoder** (compresses data) a **decoder** (reconstructs data)
 - The encoder transforms the input data into a reduced-dimensional representation, which is often referred to as “**latent space**” or “encoding”
 - From that representation, a decoder rebuilds the initial input.
- For the network to gain meaningful patterns in data, a process of encoding and decoding facilitates the definition of essential features.



General Architecture of Autoencoders?

- **Encoder:** Compresses input data into a lower-dimensional latent representation.
- **Latent Space (Bottle neck):** Encodes essential features of input data.
- **Decoder:** Reconstructs the original input from the latent space.
- **Loss Function:** Measures reconstruction error (e.g., Mean Squared Error).



General Architecture of Autoencoders?

The **encoder** is a set of convolutional blocks followed by pooling modules that compress the input to the model into a compact section called the bottleneck (the final hidden layer)



Only the most important information can pass through the **bottleneck**, which limits the data flow from the encoder to the decoder. We may say that the bottleneck aids in the creation of a knowledge representation of the input since it is built in such a way that it captures all of the information that an image possesses.



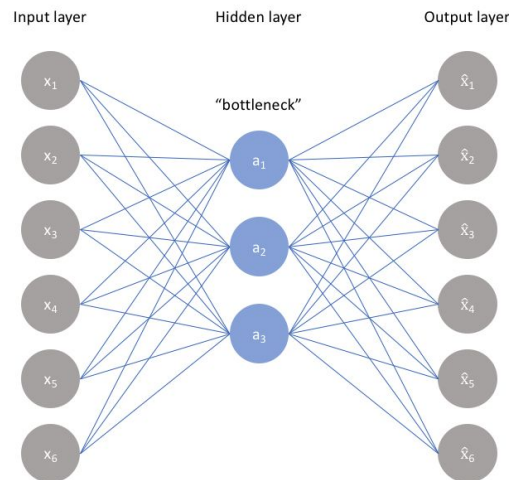
The **decoder** reconstructs the output of the bottleneck using a sequence of upsampling and convolutional blocks. The decoder functions as a “decompressor” and recreates the image from its latent properties because it receives a compressed knowledge representation as input.



Deterministic Autoencoders

What Makes an Autoencoder Deterministic?

- Encoder maps input deterministically to a single latent representation.
- Uses Mean Squared Error (MSE) or Binary Cross Entropy (BCE) loss.
- No constraints on the structure of latent space.
- Limitation: Cannot generate diverse outputs.
 - Linear Autoencoders
 - Deep Autoencoders
 - Sparse Autoencoders
 - Denoising Autoencoders



Linear Autoencoders

- A simple autoencoder where both the encoder and decoder use only linear transformations (i.e., no activation functions like ReLU or sigmoid).
- It essentially performs Principal Component Analysis (PCA) but using a neural network structure.
- The learned latent space is equivalent to PCA's projection in lower dimensions.
- **Use case:** When you want to reduce dimensionality while maintaining interpretability similar to PCA. A simple autoencoder where both the encoder and decoder use only linear transformations (i.e., no activation functions like ReLU or sigmoid).

Deep Autoencoders

- Uses multiple hidden layers in both the encoder and decoder, making it a deep neural network.
- Non-linear activation functions (e.g., ReLU, sigmoid) allow it to learn more complex hierarchical representations.
- More powerful than linear autoencoders and PCA because it can capture non-linear dependencies in data.
- **Use case:** Deep feature extraction, dimensionality reduction, or as a pretraining step for deep learning tasks.

Sparse Autoencoders

- Encourages a sparse latent representation, meaning only a few neurons in the bottleneck layer are activated at a time.
- Achieved using L1 regularization (Lasso) or a sparsity constraint on neuron activations.
- Helps in learning important features while ignoring noise or irrelevant information.
- **Use case:** Feature selection, learning interpretable representations, and applications where only a few key factors explain the data (e.g., biological data, anomaly detection).

Denoising Autoencoders

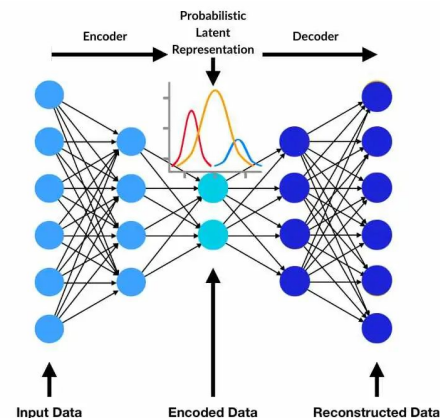
- Trained to remove noise from input data.
- During training, the input data is intentionally corrupted (e.g., adding Gaussian noise, masking pixels), but the autoencoder must reconstruct the clean version.
- Helps learn robust representations that generalize well.
- **Use case:** Image denoising, speech enhancement, and learning robust feature representations.
 - Denoising autoencoders can be used as a form of data augmentation, the restored images can be used as augmented data thus generating additional training samples



Variational Autoencoders

Why Variational Autoencoders?

- Introduce a probabilistic approach to learning representations.
- **Encoder**: Compresses input data x into a latent representation z , learning a probability distribution $q(z | x)$
- **Latent Space**: Instead of a single vector, we represent each input as a distribution with mean μ and standard deviation σ
- **Decoder**: Reconstructs the original data from a sample drawn from $q(z | x)$
- Enable sampling of new data points from learned latent space.



Why Variational Autoencoders?

- In VAEs, we don't learn a single deterministic latent vector but a distribution (mean μ and standard deviation σ).
- However, sampling directly from this distribution is not differentiable, so we cannot backpropagate gradients.
 - Solution: We use the **Reparameterization Trick**, which allows gradient flow by introducing a helper variable: $z = \mu + \sigma \cdot \epsilon$ where ϵ is sampled from a normal distribution $N(0, 1)$
 - The **Reparameterization Trick makes sampling differentiable**
 - This reformulation allows gradients to pass through μ and σ , enabling training.

Loss Function of VAEs

- The loss function in a VAE consists of two terms:
 - **Reconstruction Loss (Binary Cross-Entropy or Mean Squared Error)**
 - Ensures that the output image is similar to the input.
 - Measures how well the decoder reconstructs the input.
 - **KL Divergence (Regularization Term)**
 - Encourages the latent space to follow a normal distribution $N(0, 1)$.
 - Ensures smoothness and prevents overfitting.
- The final loss function: $L = \text{Reconstruction Loss} + \beta \cdot D_{KL}(q(z|x) || p(z))$
- **The loss function balances reconstruction accuracy and latent space regularization.**

A series of white, stylized lines on the left side of the slide, resembling a circuit board or a series of steps. The lines are of varying lengths and are arranged in a way that creates a sense of depth and movement.

Applications

Applications of Autoencoders & VAEs

1. Dimension Reduction:

Dimensionality reduction techniques include the use of linear and deep autoencoders.

2. Image Denoising:

For effective and extremely accurate image denoising, autoencoders like the denoising autoencoder can be utilized. Autoencoders, in contrast to conventional denoising techniques, extract the image from the noisy data that has been supplied to them by learning a representation of it. After that, the representation is decompressed to provide a picture without noise. Hence, denoising autoencoders can denoise complicated images that are impossible to denoise using conventional techniques.

Applications of Autoencoders & VAEs

3. Anomaly Detection

Consider an autoencoder that has been trained on a specific dataset P . For any image sampled for the training dataset, the autoencoder is bound to give a low reconstruction loss and is supposed to reconstruct the image as is. For any image which is not present in the training dataset, however, the autoencoder cannot perform the reconstruction, as the latent attributes are not adapted for the specific image that has never been seen by the network. As a result, the outlier image gives off a very high reconstruction loss and can easily be identified as an anomaly with the help of a proper threshold.

Implementation

Let's look at the implementation in
Lab 4



Project Brainstorming

Examples of Tasks

At the end of this semester, you will be able to implement projects that cover the following tasks:

1. Convolutional Neural Networks (CNNs)

- Detect pneumonia from chest X-rays.
- Build a trash-sorting vision system (plastic vs. glass vs. paper).
- Segment road lanes for self-driving car demos.
- Restore old, blurry photos into clearer images.

2. Generative Adversarial Networks (GANs)

- Generate realistic human faces that don't exist.
- Turn black-and-white photos into color.
- Improve satellite image resolution for environmental monitoring.
- Create AI-generated artwork mimicking Van Gogh.

3. Autoencoders

- Compress sensor data for IoT devices.
- Remove noise from corrupted audio recordings.
- Detect fraudulent credit card transactions (anomalies).
- Reconstruct missing parts of old manuscripts or images.

Examples of Tasks

At the end of this semester, you will be able to implement projects that cover the following tasks:

4. Recurrent Neural Networks (RNNs)

- Predict the next word in a sentence (basic text generation).
- Forecast electricity demand or weather temperature trends.
- Analyze patient heart-rate time series to flag abnormalities.
- Transcribe spoken digits into text.

5. Probabilistic Graphical Models (PGMs)

- Predict whether a student passes a course given prior grades and attendance (Bayesian network).
- Model disease spread across communities (Markov Random Fields).
- Infer missing data in survey responses.
- Create a probabilistic spam filter for emails.

6. Transformers

- Summarize research papers into short abstracts.
- Translate speech or text between English and Spanish.
- Power a chatbot that answers FAQs for a campus website.
- Match captions to the correct images (vision-language alignment).

Examples of Tasks

At the end of this semester, you will be able to implement projects that cover the following tasks:

7. Advanced Generative Models & Multimodal Architectures

- Generate images from text prompts (“a fox riding a skateboard at night”).
- Produce video summaries with both visuals + captions.
- Build a multimodal tutor: students upload an image of math work, AI gives text feedback.
- Use diffusion models to create ultra-realistic photos of objects.

8. Model Compression & Edge AI

- Deploy a bird-call classifier on a microcontroller for conservation projects.
- Run real-time gesture recognition on a Jetson Nano with a webcam.
- Optimize a face-detection model to fit on a mobile phone.
- Compare energy usage of cloud vs. edge AI for the same model.

Examples of Tasks

At the end of this semester, you will be able to implement projects that cover the following tasks:

9. Graph Neural Networks (GNNs)

- Predict friendships in a social network (link prediction).
- Classify proteins by their molecular graph structure.
- Recommend collaborators in a research community network.
- Detect fraud rings in transaction networks.

10. Modern Recommendation Systems

- Build a personalized Netflix-style movie recommender.
- Suggest products in an e-commerce dataset.
- Recommend study resources to students based on prior activity.
- Create music playlists tailored to mood and listening history.

Think about ideas and be prepared to share!